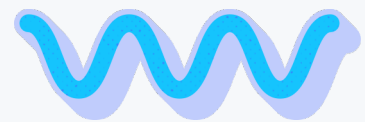




FREELANCE HUB

PRESENTATION





OUR TEAM



กัณฑ์วิชญ์ นาคนวล
673380027-4



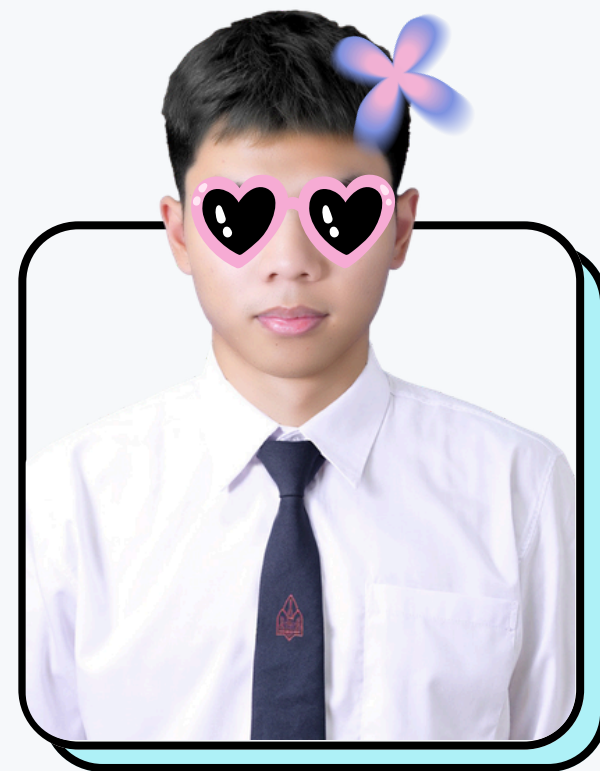
ธีรวัฒน์ อุดมจิณา
673380039-7



เพชรภิญโญ ธนศิรินรากร
673380073-7



กรมภักฐ์ พิริยะะ
673380262-4



ณัฏฐดนัย สาริกา
673380511-9





PROBLEM

ข้อมูลกระจาย

ต้องค้นหาไฟล์และรายละเอียดลูกค้า
จากหลายที่

จัดลำดับงานยาก

เมื่อรับหลายงานพร้อมกัน อาจหลุด
เดดไลน์หรือลืมสิ่งที่ลูกค้าขอแก้

เสียเวลากับงานจัดการ

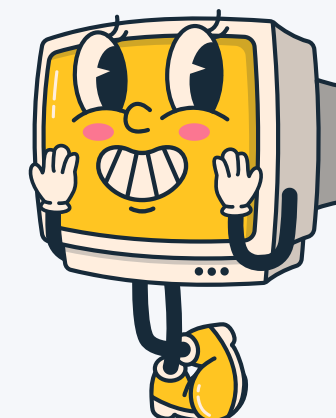
ต้องอัปเดตข้อมูลซ้ำหลายเครื่องมือ จน
เบียดเวลาทำงานหลัก

&

TARGET

ฟรีแลนซ์

ทำงานคนเดียวและรับหลายโปรเจกต์
พร้อมกัน



แก้ไขข้อมูลลูกค้า

ชื่อผู้ติดต่อ *

๒๕๖1๐1

อีเมล *

jojo@gmail.com

ที่อยู่ *

11/2/3

อำเภอ / เขต *

เมืองขอนแก่น

รหัสไปรษณีย์ *

40000

หมายเลข

ชื่อบริษัท *

CPKKU

โทรศัพท์ *

0147852369

จังหวัด *

ขอนแก่น

ตำบล / แขวง *

ในเมือง

เลขประจำตัวผู้เสียภาษี *

1234567899999

- 1.สมัครสมาชิก เข้าสู่ระบบ และจัดการโปรไฟล์
- 2.จัดการลูกค้า โปรเจกต์ และงานย่อย
- 3.จับเวลาและบันทึกเวลาทำงานย้อนหลัง
- 4.ดู DASHBOARD สรุปเวลาและความคืบหน้างาน
- 5.ดู REPORTS แยกเวลาตามลูกค้าหรือโปรเจกต์
- 6.กรองข้อมูล และส่งออก CSV

LAYERED

ARCHITECTURE

3

Presentation Layer

Controller/Rest API
(Receive HTTP req and return res)

Service Layer

Service
(Business Logic, Transaction)

Domain, DTO, Mapper

Entities Enums DTO Mapper

Repository Layer

Spring Data JPA
(Read/Write data)

Database

PostgreSQL

SOLID

4

Single Responsibility

แยกการรับ API กฎการทำงาน การเข้าถึงฐานข้อมูล และการแปลงข้อมูลออกจากกัน
เช่น

คลาส	หน้าที่
ProjectController	รับคำขอ HTTP และส่ง Response
ProjectServiceImpl	ตรวจสอบและดำเนินการตามกฎของโปรเจกต์
ProjectRepository	อ่านและบันทึกข้อมูลโปรเจกต์
ProjectMapper	แปลงข้อมูล Entity เป็น DTO

Open/close

ออกแบบให้เพิ่มพฤติกรรมใหม่ได้ โดยลดการแก้ไขโค้ดหลักเดิม
เช่น เพิ่มการจัดการ Error ได้โดยลดความเสี่ยงที่จะไปกระทบการจัดการ Error เดิม

คลาส	หน้าที่
ErrorHandler	กำหนดรูปแบบที่ตัวจัดการ Error ต้องทำตาม
ValidationErrorHandler	จัดการ Error จากข้อมูลไม่ผ่าน validation
AuthenticationErrorHandler	จัดการ Error เกี่ยวกับการยืนยันตัวตน
ErrorHandlerChain	เรียกตัวจัดการตามลำดับจนพบตัวที่รับผิดชอบ

Liskov Substitution

คลาสที่ทำตามสัญญาเดียวกัน ต้องใช้แทนกันได้ โดยผู้เรียกยังทำงานได้ถูกต้อง
เช่น Project ใช้ State แต่ละตัวผ่านรูปแบบเดียวกันได้ โดยไม่ต้องเขียนวิธีเรียกใหม่สำหรับทุกสถานะ

คลาส	หน้าที่
ProjectState	กำหนดสัญญาร่วม ได้แก่ status(), canTransitionTo(), canTrackTime() และ canEditTasks()
PlannedState	กฎของโปรเจกต์ที่อยู่ระหว่างวางแผน
ActiveState	กฎของโปรเจกต์ที่กำลังดำเนินการ
OnHoldState	กฎของโปรเจกต์ที่พักงาน
CompletedState	กฎของโปรเจกต์ที่เสร็จสิ้น
ArchivedState	กฎของโปรเจกต์ที่จัดเก็บถาวร

Interface Segregation

ผู้เรียกควรพึ่งเฉพาะเมธอดที่จำเป็นต่อการใช้
งาน เช่น Controller ต้องการแค่ ID ของผู้ใช้ จึง
ไม่จำเป็นต้องพึ่งเมธอดจัดการผู้ใช้ทั้งหมด

คลาส	หน้าที่ (ในตัวอย่างนี้)
CurrentUserProvider	กำหนดเมธอด currentUserId() เพื่ออ่าน ID ของผู้ใช้ที่เข้าสู่ระบบ
UserService	implements CurrentUserProvider และเขียนการทำงานจริง ของ currentUserId()
ClientController	รับ CurrentUserProvider มาใช้ เพื่อรู้ว่าผู้ใช้คนไหน กำลังเรียก API

Dependency Inversion Principle

ส่วนที่เรียกใช้งานพึ่ง Interface แทนการผูกกับคลาสที่
ทำงานจริงโดยตรง
เช่น ClientController รู้เพียงว่า ClientService มีความ
สามารถอะไรให้เรียก ไม่จำเป็นต้องรู้รายละเอียดว่า
ClientServiceImpl ตรวจสอบไขหรือจัดการข้อมูล
อย่างไร

คลาส	หน้าที่
ClientService	รับคำขอ API แล้วเรียก งานผ่าน ClientService
ClientServiceImpl	กำหนดความสามารถใน การจัดการ Client เช่น สร้าง อ่าน และแก้ไขข้อมูล
ClientController	implements ClientService และ ทำงานตามกฎของ Client จริง

DESIGN

PATTERNS

• PROJECT STATE

แยกกฎการทำงาน ตามสถานะ
โปรเจกต์

ProjectState ActiveState /
ArchivedState Active

จับเวลาได้ Archived จับเวลา
ไม่ได้

• TIME TACKING OBSERVER

ส่งเหตุการณ์ให้ตัวรับ ทำงานต่อ
อย่างแยกกัน

TimerServiceImpl
TimerStoppedProgressListener

หยุด timer แล้วส่ง event ให้
listener ตรวจสอบความคืบหน้า

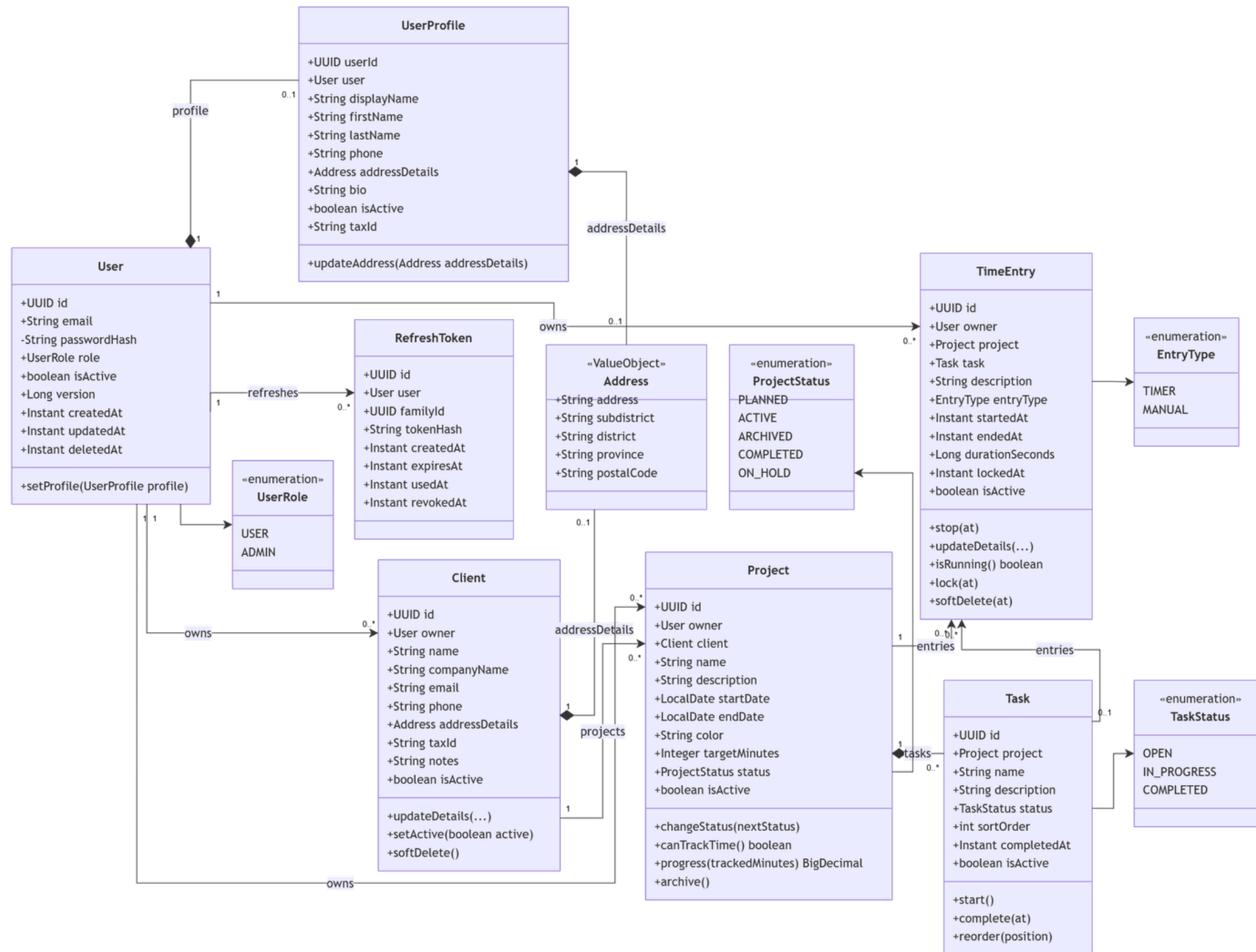
• AUTHENTICATION STRATEGY

แยกวิธีแฮชรหัสผ่าน ผ่าน
interface

PasswordEncoder
BCryptPasswordEncoder

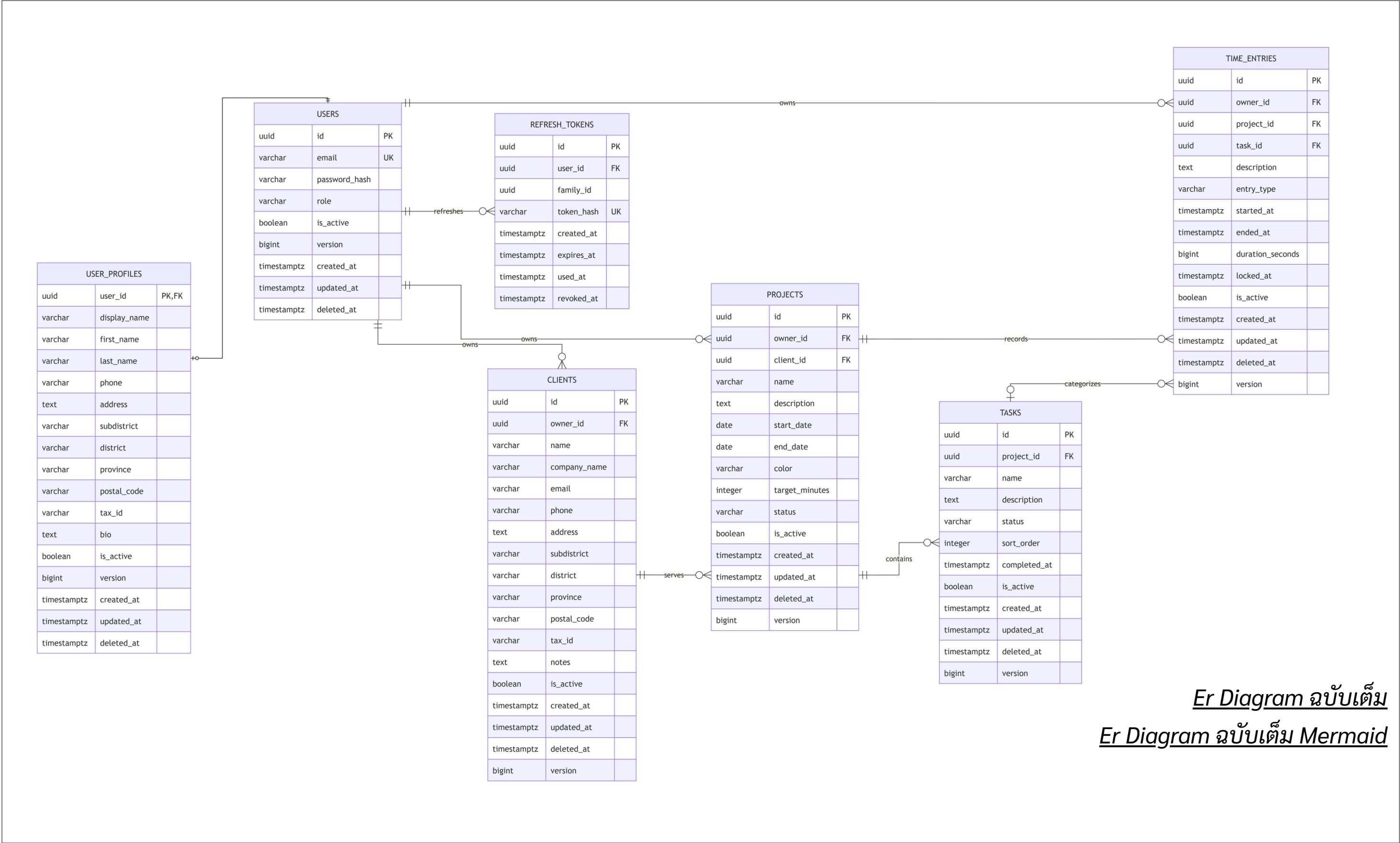
AuthServiceImpl เรียก encoder
ปัจจุบันเลือกใช้ BCrypt

CLASS DIAGRAM



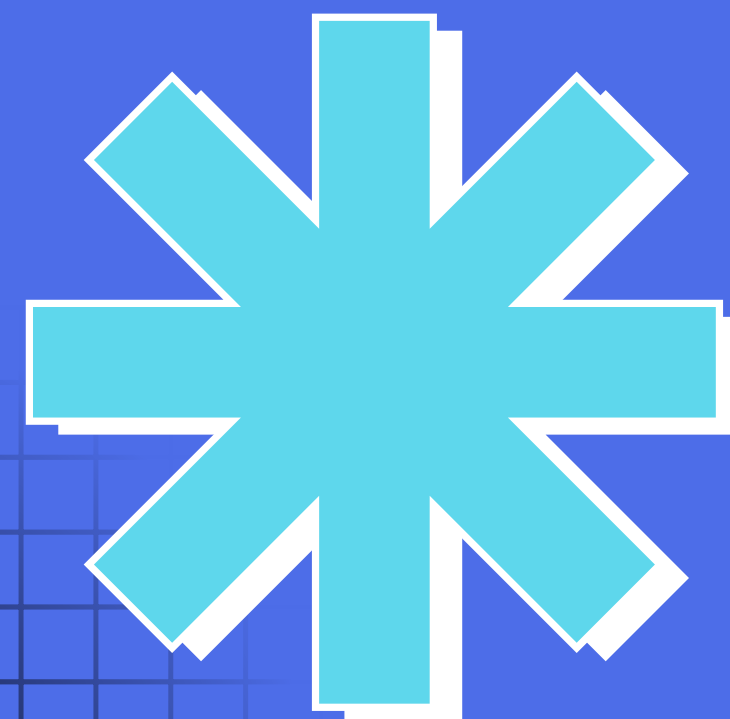
Class Diagram ฉบับเต็ม
Class Diagram ฉบับเต็ม Mermaid

ER DIAGRAM



Er Diagram ฉบับเต็ม
Er Diagram ฉบับเต็ม Mermaid

• LIVE



DEMO



freelance-hub-self.vercel.app

Lessons Learned



- การทำงานร่วมกันต้องตกลง API ให้ชัดเจน
รูปแบบ response และ pagination ต้องตรงกัน เพื่อให้หน้าบ้านเรียกใช้งานได้ถูกต้อง
- Archive กับ Soft Delete เป็นคนละเรื่อง
ต้องแยกการเปลี่ยนสถานะออกจากการลบ และทดสอบผลกระทบต่อ Client, Project และ Timer
- ต้องทดสอบทั้งกรณีปกติและกรณีที่อาจเกิดปัญหา
เช่น ไม่มีข้อมูล เลือกช่วงวันที่ต่างกัน หรือพยายามดูข้อมูลของผู้ใช้อื่น เพื่อให้มั่นใจว่าระบบทำงานถูกต้อง
- การทำ CI/CD ควรมีการแยก Staging และ Production เพื่อป้องกันการ fix on production

สิ่งที่ปรับปรุงต่อ



- เพิ่มประสิทธิภาพการดึงข้อมูล และดึงเฉพาะข้อมูลที่จำเป็น
- ปรับประสบการณ์ผู้ใช้ แสดงสถานะกำลังโหลด ไม่มีข้อมูล และข้อผิดพลาดให้ชัดเจน
- เพิ่ม feature invoice สำหรับช่วยเหลือในเรื่องการเงิน

THANK
YOU!

FREELANCE-HUB-SELF.VERCEL.APP